



FINAL PROJECT REPORT

Islamabad Ambulance Dispatch and Navigation System

Course:

CSC211 - Data Structures and Algorithms (SP26)

Instructor: Sir Ateeq Zafar

Group Member	Registration No.	Module
M.Zaeem Sajjad	BSE-039	City Graph implementation
Aneeq Abid	BSE-042	Navigation Engine and Dijkstra
Shahzil Ejaz	BSE-047	Ambulance Dispatch Queues management
Zaryab Zahid	BSE-056	Min/Max Heap, Incident BST, Traffic BST, Linklists

COMSATS University Islamabad

Department of Computer Science

Table of Contents

1. Project Introduction	3
1.1 Purpose and Goals	3
1.2 Real-World Relevance	3
1.3 Project Scope	3
2. Problem Statement	4
2.1 Core Problem	4
2.2 Secondary Problems	4
2.3 Assumptions and Limitations	4
3. Data Structures Used	5
3.1 Justification of Key Choices	5
3.1 Graph (Adjacency List)	5
3.2 AVL Binary Search Tree (per Intersection)	6
3.3 Linked List (inside BST Nodes)	6
3.4 Min-Heap (Dijkstra Frontier)	6
3.5 Max-Heap (Global Alert System)	7
3.6 Circular Queue (News Ticker) & Patient Queue	7
4. System Design and Data Object Description	8
4.1 Architecture Overview	8
4.2 Key Data Object Descriptions	8
GraphNode (Graph.cpp)	8
IncidentReport (DataTypes.cpp)	8
PatientCall (DataTypes.cpp)	9
5. Functional Specifications	10
5.1 computeEdgeCost(toID, baseDist) - NavigationEngine	10
5.2 runDijkstra(src, dst) - NavigationEngine	10
5.3 AVL Insert and Range Query - IncidentBST	10
5.4 addCall() and extractNext() - PatientQueue	11
6. Code Snapshots with Explanation	12
6.1 Smart Edge Cost Formula (NavigationEngine.cpp)	12
6.2 AVL Tree Insertion with Auto-Balancing (IncidentBST.cpp)	12
6.3 Dijkstra Core Loop (NavigationEngine.cpp)	13
6.4 Ambulance Dispatch Simulation (DispatchSim.cpp)	14
7. Testing and Sample Output	15
7.1 Test Case: Danger-Aware Routing (Node 0 to Node 49)	15
7.2 Test Case: Patient Queue Ordering	15
7.3 Test Case: Edge Case - Invalid Node Input	16
7.4 Test Case: AVL Range Query	16
8. Conclusion	17
8.1 Summary of Key Learnings	17
8.2 Benefits of the Approach	17
8.3 Limitations	17
8.4 Future Improvements	17
9. References	19
10. Appendix: Full Module Reference	19
10.1 File Structure	19
10.2 Node Map (50 Islamabad Locations)	19

1. Project Introduction

The Islamabad Ambulance Dispatch and Navigation System is a C++ console application developed as the final project for CSC211 Data Structures and Algorithms at COMSATS University Islamabad (Spring 2026). The system simulates a real-world emergency dispatch center rooted at PIMS Hospital, capable of intelligently routing ambulances across a 50-node graph of actual Islamabad landmarks.

1.1 Purpose and Goals

- Compute the safest and fastest ambulance route from PIMS Hospital to any patient location using a danger- and traffic-aware shortest-path algorithm.
- Maintain a live patient call queue prioritized by medical urgency, ensuring the most critical cases are attended first.
- Store, query, and display road incident reports and live traffic congestion data per intersection using efficient tree structures.
- Demonstrate practical application of multiple data structures working together in a single real-world simulation.

1.2 Real-World Relevance

Emergency dispatch systems depend on fast, accurate routing under dynamic conditions. A standard shortest-path algorithm fails in emergency scenarios because it ignores road incidents (accidents, gas leaks, floods) and live traffic congestion. This project addresses that gap by augmenting Dijkstra's algorithm with real-time hazard and congestion data stored in per-node AVL trees, mirroring approaches used in modern GPS and dispatch systems.

1.3 Project Scope

- 50-node undirected weighted graph modelling real Islamabad intersections and roads.
- File-driven initialization: road edges, node names, incident reports, and traffic records all loaded from text files.
- Full menu-driven console interface with 14 operational commands.
- Simulation of ambulance dispatch, patient pickup, and vehicle return trips.
- No third-party libraries used; all data structures are custom implemented from scratch.

2. Problem Statement

Standard emergency dispatch systems face a critical limitation: routing algorithms typically minimize only raw geographic distance, completely ignoring dynamic hazards such as road accidents, flooding, gas leaks, and traffic congestion. In a city like Islamabad, with heavy traffic corridors and frequent incidents at key junctions (Blue Area, Zero Point, Faizabad Interchange), a shortest-distance route may actually be the most dangerous or slowest route at the time of dispatch.

2.1 Core Problem

Given a city modelled as a weighted undirected graph, and given real-time incident severity scores and traffic congestion multipliers attached to every intersection, how do we compute the optimal path for an ambulance that minimizes both travel time and hazard exposure simultaneously?

2.2 Secondary Problems

- How do we manage multiple simultaneous patient calls fairly, ensuring the highest-urgency patient is always served first?
- How do we efficiently store, retrieve, and range-query time-stamped incident and traffic records per intersection, especially as new data keeps arriving?
- How do we maintain active danger alerts with automatic expiry while always surfacing the most severe alert on demand?

2.3 Assumptions and Limitations

- The graph is undirected (roads are traversable in both directions).
- All ambulances begin at Node 0 (PIMS Hospital).
- Patient location is identified by graph node ID; no GPS coordinates are used.
- Time-stamped data uses a 24-hour window (now - 86400 to now + 3600) for AVL tree range queries.
- Simulation is console-based; no real-time data feed or GUI is implemented.

3. Data Structures Used

The project uses six distinct custom-implemented data structures, each chosen for a specific purpose within the system. Together they form a layered architecture where the graph holds per-node trees, and the trees hold per-timestamp linked lists.

Data Structure	Location	Purpose
Adjacency Linked List	Each GraphNode	Stores outgoing road edges (to-node, distance)
AVL BST (IncidentBST)	Each GraphNode	Time-ordered incident reports with range-sum query
AVL BST (TrafficBST)	Each GraphNode	Time-ordered traffic readings with range-avg query
Singly Linked List (IncidentList / TrafficList)	Inside BST nodes	Chains same-timestamp records per BST node
Min-Heap	NavigationEngine	Priority queue driving Dijkstra (lowest cost first)
Max-Heap (AlertHeap)	NavigationEngine	Active danger alerts, highest danger at top
Max-Heap (PatientQueue)	main.cpp	Patient calls sorted by urgency (1-10)
Circular Queue (Ticker)	NavigationEngine	50-slot news ticker with overwrite-on-full

3.1 Justification of Key Choices

This project implements six distinct data structures, all coded from scratch in C++ without any STL containers. They are organised in a four-layer nested architecture:

3.1 Graph (Adjacency List)

File: graph/Graph.cpp

The city is represented as an undirected weighted graph. Each GraphNode holds an integer ID, a string name, an adjacency linked-list of Edge structs (toNode, distance, next*), and pointers to its own IncidentBST and TrafficBST. The CityGraph class manages an array of GraphNode objects and provides addEdge(), loadFromFile(), insertIncident(), and insertTraffic() methods.

Justification:

Adjacency list representation gives $O(V + E)$ traversal — ideal for sparse city graphs. Building the graph as a plain array of nodes (no STL vector) with manual new/delete demonstrates raw heap ownership.

Complexity:

$O(V + E)$ space; $O(V + E)$ traversal; $O(1)$ edge insertion.

3.2 AVL Binary Search Tree (per Intersection)

Files: data_structures/IncidentBST.cpp, data_structures/TrafficBST.cpp

Every GraphNode embeds two self-balancing AVL BSTs one for IncidentReports (keyed on Unix timestamp) and one for TrafficRecords. Each BST node stores a timestamp key and a linked list of reports sharing that timestamp (handling collision). The AVL rotation logic (rotL, rotR, balance) keeps tree height at $O(\log N)$, ensuring efficient range queries even under heavy insert load.

Justification:

The critical operation is a time-window range query fetch all incidents/traffic within the last 25 hours. A sorted BST allows pruning of irrelevant subtrees during the range scan, giving $O(\log N + K)$ complexity where K is the number of results. An unsorted list would require $O(N)$ scanning.

Complexity:

$O(\log N)$ insert; $O(\log N + K)$ range query; $O(\log N)$ height-balanced by AVL rotations.

3.3 Linked List (inside BST Nodes)

File: data_structures/LinkedList.cpp

When two incidents occur at the exact same Unix timestamp, they share a single BST node. That BST node holds a singly linked IncidentList (or TrafficList). The linked list appends new reports in $O(1)$ amortised time and provides sumSeverity() and avgCongestion() helper methods used by the BST range query.

Justification:

Timestamp collisions are uncommon but real in real-time systems. A linked list inside each BST node elegantly handles the collision case without requiring a full BST restructure, creating a true 3-level nested architecture: Graph $\rightarrow$ BST $\rightarrow$ Linked List.

3.4 Min-Heap (Dijkstra Frontier)

File: data_structures/MinHeap.cpp

A custom min-heap keyed on cumulative path cost (float) drives the Dijkstra priority queue. It stores HeapNode structs {nodeID, cost} and supports insert() in $O(\log N)$ via heapifyUp and

extractMin() in $O(\log N)$ via heapifyDown. Capacity is set to $V \times 6$ to accommodate multiple insertions of the same node as shorter paths are discovered.

3.5 Max-Heap (Global Alert System)

File: data_structures/MaxHeap.cpp

A second max-heap stores AlertNode structs {intersectionID, dangerScore, message, expiresAt}. Highest dangerScore floats to the root, giving $O(1)$ access to the city's most critical active hazard. A purgeExpired() method removes alerts whose TTL has lapsed before each display call.

Justification:

Serving two simultaneous roles the Dijkstra frontier and the global alert board — demonstrates deep heap versatility. The dual-heap design (Min for routing, Max for alerting) is an intentional architectural decision that avoids any conflict of interest between the two subsystems.

3.6 Circular Queue (News Ticker) & Patient Queue

File: data_structures/Queue.cpp

The CircularQueue is a fixed-size ring buffer of 50 string slots with manual head/tail/cnt pointer arithmetic. On the 51st enqueue, the head pointer advances to overwrite the oldest entry, implementing a zero-allocation rotating log.

The PatientQueue is a second max-heap keyed on patient urgency (1–10). It enables the dispatch simulation to always serve the highest-urgency patient first, regardless of registration order.

4. System Design and Data Object Description

4.1 Architecture Overview

The system is organized into six modules, each implemented as a separate .cpp file. The dependency chain flows from data types outward through data structures, graph, navigation, dispatch, and UI layers.

Module	File	Responsibility
Data Types	DataTypes.cpp	Plain structs: IncidentReport, TrafficRecord, PatientCall, HeapNode, AlertNode
Linked Lists	LinkedList.cpp	IncidentList and TrafficList used inside BST nodes
BST Trees	IncidentBST.cpp, TrafficBST.cpp	Per-node AVL trees for time-indexed record storage and range queries
Heap Structures	MinHeap.cpp, MaxHeap.cpp, Queue.cpp	Priority queues and circular ticker buffer
Graph	Graph.cpp	50-node city map with file loaders for edges, incidents, and traffic
Navigation	NavigationEngine.cpp	Dijkstra with smart edge cost formula, alert insertion, ticker management
Dispatch	AmbulanceFleet.cpp, DispatchSim.cpp	Ambulance linked list and full dispatch simulation flow
UI	Menu.cpp	Terminal banner, menu display, pressEnter utility

4.2 Key Data Object Descriptions

GraphNode (Graph.cpp)

Field	Type	Description
id	int	Node index (0-49)
name	string	Human-readable location name (e.g., PIMS_Hospital)
incBST	IncidentBST*	AVL tree of time-stamped incident reports for this node
trafficBST	TrafficBST*	AVL tree of time-stamped traffic records for this node
adjList	Edge*	Head pointer of outgoing road linked list

IncidentReport (DataTypes.cpp)

Field	Type	Description
incidentID	string	Unique identifier (e.g., INC001)
timestamp	long	Unix epoch time of the incident
severity	int (1-10)	Danger level; 10 = most severe
description	string	Short textual description of the incident

PatientCall (DataTypes.cpp)

Field	Type	Description
callID	int	Auto-incremented call identifier starting at 1001
patientNode	int (0-49)	Graph node where the patient is located
urgency	int (1-10)	Medical priority; heap key for PatientQueue
patientName	string	Patient identifier (no spaces)
condition	string	Medical condition description
dispatched / completed	bool	Lifecycle flags for tracking dispatch status

5. Functional Specifications

5.1 computeEdgeCost(toID, baseDist) - NavigationEngine

Purpose	Computes the weighted cost of traversing an edge to a given node, factoring in real-time incidents and traffic congestion.
Input	toID: destination node ID baseDist: raw road distance in km
Output	float: adjusted edge cost = (baseDist * congestionMult) + (dangerScore * riskMult)
Pre-condition	Graph node with id=toID must exist and have initialized incBST and trafficBST.
Post-condition	Returns a non-negative float; returns baseDist unchanged if node is null.
Algorithm	Query incBST.querySeverity(now-86400, now+3600) for danger score. Query trafficBST.queryCongestion(same window) for multiplier. Apply formula.

5.2 runDijkstra(src, dst) - NavigationEngine

Purpose	Finds the lowest total-cost path from source to destination using a custom edge cost that incorporates danger and traffic.
Input	src: source node (0-49) dst: destination node (0-49)
Output	bool: true if path found. Prints path and total cost to console.
Pre-condition	Graph must be loaded. src and dst must be valid, distinct node IDs.
Post-condition	dist[], prev[], vis[] arrays freed. finalPathNodes[] updated for search space display.
Algorithm	Initialize dist[src]=0, all others=infinity. Insert src into MinHeap. Extract min, relax all outgoing edges via computeEdgeCost. Backtrack prev[] to reconstruct path.

5.3 AVL Insert and Range Query - IncidentBST

Purpose	insert(): Add an IncidentReport keyed by timestamp, self-balancing the tree. querySeverity(): Sum severity of all incidents in a time window.
Input (insert)	IncidentReport r with a valid timestamp and severity (1-10).
Input (query)	lo, hi: Unix epoch time bounds defining the query window.
Output (query)	int: total severity score of all incidents in [lo, hi].
Algorithm	Standard AVL recursive insert with height update and rotation (LL/RR/LR/RL). rangeSum() traverses left subtree if node.timestamp > lo, right if < hi, accumulates severity if in range.

5.4 addCall() and extractNext() - PatientQueue

Purpose	addCall(): Register a new patient emergency and insert into the max-heap. extractNext(): Remove and return the highest-urgency patient.
Input (addCall)	node (0-49), urgency (1-10), name, condition, city pointer.
Output	addCall returns callID (>0) on success, -1 on error. extractNext returns PatientCall; returns default (callID=-1) if queue is empty.
Algorithm	Insert at end of heap array, heapifyUp until max-heap property restored. extractNext swaps root with last, shrinks size by 1, heapifyDown from root.

6. Code Snapshots with Explanation

6.1 Smart Edge Cost Formula (NavigationEngine.cpp)

The core innovation of the system is the `computeEdgeCost` function. Instead of using raw road distance, it queries the AVL trees of the destination node and computes an adjusted cost that penalizes dangerous and congested routes.

```
float computeEdgeCost(int toID, float baseDist) {
    GraphNode *node = graph->getNode(toID);
    if (!node) return baseDist;

    long now = (long)time(nullptr);
    long lo = now - 86400; // 24h past window
    long hi = now + 3600; // 1h future window

    int dangerScore = node->incBST->querySeverity(lo, hi);
    float congestionMultiplier = node->trafficBST->queryCongestion(lo, hi);

    float travelCost = baseDist * congestionMultiplier;
    float hazardPenalty = (float)dangerScore * riskMult;
    return travelCost + hazardPenalty;
}
```

Explanation: The function retrieves the target node's two AVL trees. It queries IncidentBST for total severity within a rolling 25-hour window, and TrafficBST for average congestion in the same window. The final cost = (base distance multiplied by congestion) + (danger score multiplied by a configurable risk multiplier). A heavily congested road with major incidents will receive a very high cost, causing Dijkstra to route around it automatically.

6.2 AVL Tree Insertion with Auto-Balancing (IncidentBST.cpp)

The AVL self-balancing mechanism ensures the BST never becomes a linear chain even when incidents arrive in chronological order.

```
IncBSTNode* ins(IncBSTNode* node, IncidentReport r) {
    if (!node) return new IncBSTNode(r.timestamp, r);
    if (r.timestamp == node->timestamp) {
        node->incidents->append(r); // same-time: add to linked list
        return node;
    }
    if (r.timestamp < node->timestamp)
        node->left = ins(node->left, r);
```

```

else
    node->right = ins(node->right, r);
return balance(node); // rotate if needed
}

IncBSTNode* rotR(IncBSTNode* y) {
    IncBSTNode* x = y->left;
    IncBSTNode* T2 = x->right;
    x->right = y; y->left = T2;
    upH(y); upH(x);
    return x;
}

```

Explanation: Insertion follows standard BST rules keyed on timestamp. If two incidents share a timestamp, they are chained in a linked list at the same BST node rather than creating a duplicate node. After every recursive insert, balance() recalculates the balance factor and applies the appropriate single or double rotation (LL, RR, LR, RL) to keep height difference at most 1.

6.3 Dijkstra Core Loop (NavigationEngine.cpp)

The main traversal loop drives the pathfinding engine.

```

while (!pq.isEmpty()) {
    HeapNode cur = pq.extractMin();    // O(log V)
    int currentNode = cur.nodeID;
    if (vis[currentNode]) continue;    // already settled
    vis[currentNode] = true;
    if (currentNode == dst) break;     // early exit

    Edge *edgeLink = graph->getNode(currentNode)->adjList;
    while (edgeLink) {
        int v = edgeLink->toNode;
        if (!vis[v]) {
            float edgeCost = computeEdgeCost(v, edgeLink->distance);
            float newDist = dist[currentNode] + edgeCost;
            if (newDist < dist[v]) {    // relaxation
                dist[v] = newDist;
                prev[v] = currentNode;
                pq.insert(HeapNode(v, newDist));
            }
        }
        edgeLink = edgeLink->next;
    }
}

```

Explanation: The loop extracts the minimum-cost unvisited node from the MinHeap. It skips already-visited nodes to handle duplicate heap entries that arise from relaxation updates. For each unvisited neighbor, it computes the adjusted edge cost, checks the relaxation condition, and pushes the updated cost onto the heap. An early break at the destination avoids unnecessary computation.

6.4 Ambulance Dispatch Simulation (DispatchSim.cpp)

The dispatch simulation orchestrates the full emergency response workflow.

```
void simulateAmbulanceDispatch(NavigationEngine *engine,
                               PatientQueue *pq,
                               AmbulanceFleet *fleet) {
    if (pq->isEmpty()) { cout << "No pending calls."; return; }

    pq->displayAll(engine->getGraph());
    PatientCall call = pq->extractNext(); // highest urgency
    Ambulance *amb = fleet->getAvailable();

    dispatchAmbulance(engine, amb, call); // PIMS -> patient

    amb->currentNode = call.patientNode;
    amb->status = "RETURNING";
    returnAmbulance(engine, amb); // patient -> PIMS
}
```

Explanation: The function extracts the most critical patient from the PatientQueue max-heap, retrieves the first available ambulance from the linked-list fleet, routes the ambulance from PIMS to the patient using runDijkstra, then automatically runs the return trip simulation. The ambulance status and availability flags are updated throughout to maintain fleet state.

7. Testing and Sample Output

7.1 Test Case: Danger-Aware Routing (Node 0 to Node 49)

This is the most comprehensive test, routing from PIMS Hospital (Node 0) to Attock Bridge (Node 49) across the full city map with all pre-loaded incidents and traffic data active.

[GRAPH] Loaded 110 edges (220 directed) into 50 nodes.

[INCIDENTS] Loaded 30 incident reports.

[TRAFFIC] Loaded 16 traffic congestion updates.

Source: 0 (PIMS_Hospital) Destination: 49 (Attock_Bridge)

[RESULT] Path cost (dist + danger + traffic): 47.23 units

Route: PIMS_Hospital(BASE)

- > G-8_Markaz
- > F-8_Markaz
- > I-8_Markaz
- > I-9_Industrial
- > Tarnol_Interchange
- > PWD_Colony
- > Golra_Mor
- > Srinagar_Highway
- > Motorway_M2_Entry
- > Wah_Cantonment
- > Attock_Bridge

Observation: The algorithm avoids Blue Area (Node 10) despite it being geographically close, because its combined danger score (INC003: severity 9, INC018: severity 8) and congestion multiplier (3.2x) make its edge cost far higher than the detour.

7.2 Test Case: Patient Queue Ordering

Three patients registered with urgency levels 5, 9, and 7. The queue correctly surfaces urgency 9 first.

[CALL 1001] Registered: Ali_Khan at Rawalpindi_Saddar | Urgency: 9/10

[CALL 1002] Registered: Sara_Malik at G-9_Markaz | Urgency: 5/10

[CALL 1003] Registered: Omar_Sheikh at Zero_Point | Urgency: 7/10

PATIENT CALL QUEUE (by urgency):

[CALL 1001] Ali_Khan | Node 25 (Rawalpindi_Saddar) | Urgency: 9

[CALL 1003] Omar_Sheikh | Node 40 (Zero_Point) | Urgency: 7

[CALL 1002] Sara_Malik | Node 2 (G-9_Markaz) | Urgency: 5

7.3 Test Case: Edge Case - Invalid Node Input

When a node ID outside the valid range (0-49) is entered, the system rejects it gracefully with an error message and returns to the menu without crashing.

```
Source node (0-49): 99
Destination node (0-49): 5
[ERROR] Invalid nodes.
```

```
Node (1-49): -5
[ERROR] Invalid node.
```

7.4 Test Case: AVL Range Query

After inserting an incident at Node 30 (Pakistan Secretariat, severity 10) with a current timestamp, querying the severity for the 25-hour window returns a value that includes that incident's contribution, causing Dijkstra to heavily penalize edges leading to Node 30.

```
[INCIDENT] Logged at Pakistan_Secretariat | Sev:10 | Bridge_CRITICAL
[INCIDENT] Logged at Pakistan_Secretariat | Sev:5 | Emergency_staging
```

```
// querySeverity(now-86400, now+3600) at Node 30 returns: 15
// edgeCost = baseDist * 2.8 (traffic) + 15 * 2.0 (risk) = high penalty
```


8. Conclusion

8.1 Summary of Key Learnings

- AVL self-balancing is essential for BSTs that receive ordered data. A plain BST degrades to $O(N)$ for timestamp-ordered insertions; the AVL tree guarantees $O(\log N)$ at all times.
- Composing data structures (BST containing linked lists, graph containing BSTs) enables rich query semantics that no single structure could provide alone.
- Dijkstra's correctness depends on non-negative edge weights. Our smart cost formula always produces non-negative values, preserving algorithm correctness while adding real-world context.
- Heap-based priority queues drastically outperform sorted arrays or linear searches for both ambulance dispatch ($O(1)$ peek) and shortest-path computation ($O(\log V)$ extract).
- Encapsulating data behind module boundaries (each .cpp file as a self-contained class) made the 14-feature system manageable and allowed parallel development.

8.2 Benefits of the Approach

- The AVL tree range query allows the system to answer 'how dangerous is this intersection right now?' in $O(\log N)$ time, which is far superior to scanning a flat array.
- The circular queue ticker provides a zero-allocation event log that never runs out of memory regardless of how long the system runs.
- The modular file structure means each data structure can be tested and debugged independently before integration.

8.3 Limitations

- The graph is static after loading; edges cannot be added or removed at runtime.
- The simulation assumes instantaneous travel; no real-time clock progression models the ambulance's journey.
- The 24-hour query window is fixed; a production system would use dynamic sliding windows per graph traversal step.
- Only one dispatch queue serves all patients; a real system would partition by geographic zone.

8.4 Future Improvements

- Integrate a real-time data feed (IoT sensors, city APIs) to populate the AVL trees dynamically.

- Add A* heuristic search to the navigation engine for faster pathfinding on larger graphs.
- Implement a graphical map visualization using a lightweight 2D rendering library.
- Add multi-hospital support so the nearest available hospital (not just PIMS) can be selected for dispatch.
- Introduce a simulation clock so travel time between nodes is modelled realistically.

9. References

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press.
2. COMSATS University Islamabad. (2026). CSC211 Lab Manual: Data Structures and Algorithms (Spring 2026).
3. GeeksforGeeks. (2024). AVL Tree | Set 1 (Insertion). <https://www.geeksforgeeks.org/avl-tree-set-1-insertion/>
4. cppreference.com. (2024). std::time, std::priority_queue. <https://en.cppreference.com/>
5. Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. Numerische Mathematik, 1(1), 269-271.

10. Appendix: Full Module Reference

10.1 File Structure

```
Project_FINAL_1/
|-- main.cpp          Entry point and menu handlers
|-- data/
| |-- graph.txt       50 nodes, 110 bidirectional edges
| |-- incidents.txt   30 pre-loaded incident reports
| |-- traffic.txt     16 traffic sensor records
|-- data_structures/
| |-- DataTypes.cpp   Plain structs (no logic)
| |-- LinkedList.cpp  IncidentList, TrafficList
| |-- IncidentBST.cpp AVL tree for incidents
| |-- TrafficBST.cpp  AVL tree for traffic
| |-- MinHeap.cpp     Dijkstra priority queue
| |-- MaxHeap.cpp     Alert danger heap
| |-- Queue.cpp       CircularQueue + PatientQueue
|-- graph/
| |-- Graph.cpp       CityGraph, Edge, GraphNode
|-- navigation/
| |-- NavigationEngine.cpp Dijkstra + cost formula + alerts
|-- dispatch/
| |-- AmbulanceFleet.cpp Ambulance struct + linked list
| |-- DispatchSim.cpp   Full dispatch simulation
|-- ui/
| |-- Menu.cpp         Terminal banner and menu
```

10.2 Node Map (50 Islamabad Locations)

ID	Location	ID	Location	ID	Location
0	PIMS_Hospital (BASE)	17	E-7_Sector	34	H-11_Sector
1	G-8_Markaz	18	E-11_Sector	35	Golra_Mor
2	G-9_Markaz	19	I-10_Sector	36	Koral_Chowk

3	F-10_Markaz	20	Tarnol_Interchange	37	Nurpur_Shahan
4	F-8_Markaz	21	G-7_Sector	38	Bari_Imam
5	F-7_Markaz	22	Margalla_Road	39	Diplomatic_Enclave
6	G-10_Markaz	23	D-12_Sector	40	Zero_Point
7	F-11_Markaz	24	E-12_Sector	41	Shaheed-e-Millat_Rd
8	I-8_Markaz	25	Rawalpindi_Saddar	42	Jinnah_Avenue
9	G-11_Markaz	26	Bahria_Town_Ph1	43	Islamabad_Highway
10	Blue_Area	27	PWD_Colony	44	Srinagar_Highway
11	F-6_Supermarket	28	Bhara_Kahu	45	Rawalpindi_Ring_Rd
12	Aabpara	29	Saidpur_Village	46	Motorway_M2_Entry
13	I-9_Industrial	30	Pakistan_Secretariat	47	Taxila_Road
14	G-6_Sector	31	Shakarparian	48	Wah_Cantonment
15	Faizabad_Interchange	32	H-8_Sector	49	Attock_Bridge
16	F-5_Markaz	33	H-9_Sector	-	

Dry Run

